Lab 2: Convolutional Neural Networks

Computing layer parameters by hand, transfer learning on images, and patient-wise evaluation on ECG

Deep Learning Course

1 Overview

In this lab you build convolutional networks for two kinds of data, and in both you do most of the work with pen and paper: you work out which numbers go into the layers, and the code tells you whether you were right. You do not write much code.

- **Exercise 1 (images, 2D).** Compute the layer parameters of LeNet-5 for colour images by hand, then adapt a pretrained ResNet18 to CIFAR-10: skip its last stage, add your own output layer, and train only that layer.
- **Exercise 2 (ECG, 1D).** Compute the layer parameters of a 1D residual CNN for heartbeats, split the data *per patient*, and evaluate with precision, recall and F1 instead of accuracy alone. A second run with a wrong, random split shows how leakage overestimates the performance.

By the end you should be able to:

- compute the output shape and the number of parameters of Conv, pooling and Linear layers by hand;
- adapt a pretrained network: drop a stage, replace the head, freeze the rest;
- explain why data must be split per subject, and what leakage does to a validation score;
- read precision, recall and F1 per class, and explain why accuracy alone can mislead.

2 Setup and workflow

Everything runs on the cluster with the course modules (not conda). Load the PyTorch module before running any script:

```
module load PyTorch-bundle/2.9.1-foss-2025a-CUDA-12.8.0-whl
```

The datasets and the pretrained ResNet18 weights are on the shared staging storage (read-only) in `/staging/leuven/stg_00220/Labs/Lab2/`. Nothing is downloaded.

Each exercise is a folder with numbered scripts. Every script is a standalone program: it reads what the previous step saved to disk, does one job, and saves its own output (`processed/`, `artifacts/`, `plots/`). Run them in order. Step 01 of each exercise checks that the module is loaded.

First on the CPU, then on the GPU

The training scripts (steps with `DEVICE` at the top) work in two stages:

1. `DEVICE = "cpu"` (default): trains on a small subset for 2 epochs. It only checks that your code runs and that the shapes are right. Nothing is saved.

2. When it works, change it to `DEVICE = "cuda"` and run it on a GPU node: full training (20 epochs), results are saved.

The script prints which device it uses. If you ask for `"cuda"` on a node without a GPU, it tells you so and falls back to the CPU (and training is slow).

Where you write code

Look for `TODO` in the files. Replace every `None` by a number. Each TODO is only a few lines.

3 Background: shapes and parameters

Convolution and pooling

For a Conv layer with kernel size k, padding p and stride s on an input of length n (the height or the width for 2D, the time axis for 1D):

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

A pooling layer uses the same formula, with its own kernel size and stride (by default the stride of `MaxPool` equals its kernel size). The number of output channels is the `out_channels` of the Conv layer, and pooling does not change the number of channels.

Parameters

A `Conv2d(in_channels, out_channels, kernel_size)` has out_channels $\times$ (in_channels $\times$ $k \times k + 1$) parameters: one filter of size in_channels $\times$ $k \times k$ per output channel, plus one bias each. A `Conv1d` is the same with k instead of $k \times k$. A `Linear(in_features, out_features)` has out_features $\times$ (in_features $+ 1$) parameters. Pooling, ReLU and Flatten have none. After `Flatten`, the number of features is channels $\times$ height $\times$ width (2D) or channels $\times$ length (1D).

Residual block

A residual block (Exercise 1's ResNet18 and Exercise 2) computes two convolutions and *adds* the block's input to their output (the skip connection). For the addition to work, the output must have the same shape as the input, so the convolutions use padding.

Precision, recall and F1

For one class c, with TP, FP and FN the true positives, false positives and false negatives:

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Precision answers "of the beats the model calls c, how many really are c?" and recall answers "of the beats that really are c, how many does it find?". The *macro* F1 is the average of the F1 of all classes, so every class counts equally, however rare.

4 Exercise 1: images (2D)

Data: CIFAR-10, 60,000 colour images of 32×32 pixels in 10 classes. The training images are split into 45,000 for training and 5,000 for validation; the official 10,000 test images are only used at the end.

Folder: `exercise1_image_2d/`

Step 1: `01_prepare_data.py`

Checks the environment and the data, shows the shapes and the class balance, and saves a plot of example images (`plots/examples_per_class.png`). It also computes the mean and standard deviation of each colour channel on the training images and saves the train/validation split. Nothing to fill in.

Step 2: LeNet-5 for RGB images (`02_train_scratch_cnn.py`)

Your task

Implement the LeNet-5 architecture from the slide on *RGB* images. The `nn.Sequential` is given with every layer argument set to `None`. Work out each number by hand from the figure and fill it in.

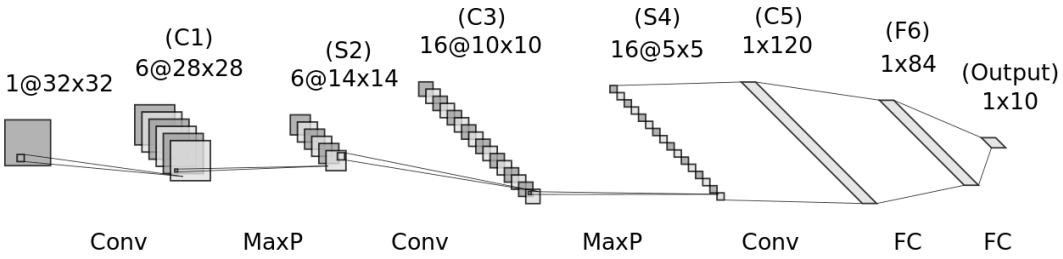


Figure 1: LeNet-5. Figure: Daniel Voigt Godoy (https://github.com/dvgodoy/dl-visuals), via Wikimedia Commons (https://en.wikipedia.org/wiki/LeNet), licensed CC BY 4.0.

Differences between the figure and this exercise. The figure shows one input channel (grayscale), ours has 3 (RGB), so the first convolution has 3 input channels. And C5 is drawn as a convolution that outputs 120 values; we use a `Flatten` followed by a `Linear` layer, which gives the same result. The other sizes are the ones you need.

The shape after each layer should be:

Layer	Output shape (channels × height × width)
input	$3 \times 32 \times 32$
C1: Conv2d, ReLU	$6 \times 28 \times 28$
S2: MaxPool2d	$6 \times 14 \times 14$
C3: Conv2d, ReLU	$16 \times 10 \times 10$
S4: MaxPool2d	$16 \times 5 \times 5$
Flatten	a vector of _____ values (you compute it)
C5: Linear, ReLU	120
F6: Linear, ReLU	84
output: Linear	10

The script prints the shape after every layer: compare it with the table. A wrong number crashes at that layer. Note that the images have 3 channels, not 1 as in the original LeNet-5. The first (given) layer normalizes the images.

Question

Compute by hand the number of parameters of the first Conv2d and of the first Linear layer. Does the total printed by the script add up?

Run it with `DEVICE = "cpu"` first, then with `"cuda"`.

Step 3: pretrained ResNet18 without its last stage

ResNet18 is a stem (`conv1`, `bn1`, `relu`, `maxpool`) followed by four stages (`layer1` to `layer4`), an average pool and a fully connected layer (`fc`). The early layers detect general patterns (edges, colours, textures), the last stage is large and specialised for the 1000 ImageNet classes. You keep the network up to `layer3` and add your own `fc`.

First run `inspecting_resnet18.py` and read its output: the layer names (section 1), the number of channels after each layer (section 5) and the code of ResNet's own `forward` (section 7).

Your task

In `mini_resnet.py`, complete `MiniResNet18`, a subclass of `nn.Module`: (1) create a new `fc` with the right number of input features, and (2) write the `forward` so that it goes through the stem, `layer1`, `layer2` and `layer3`, skipping `layer4`, then the average pool, a flatten and your `fc`. Test it with `python mini_resnet.py`.

(`layer4` is replaced by `nn.Identity()` for you, only to keep the parameter counts accurate.)

Your task

In `03_train_pretrained.py`, freeze the whole network *except* the last `fc` layer. The script prints the number of trainable parameters; you should get 2,570.

The images are normalized as ImageNet images and upsampled from 32×32 to 64×64, which is closer to what the pretrained filters have seen. Run on the CPU first, then on the GPU.

Question

Why is only 2,570 of about 2.8 million parameters trained? Why can this still work well?

Step 4: `04_compare_and_evaluate.py`

Plots the training curves of both models, evaluates them once on the test set, and shows a confusion matrix per model. Which classes are confused, and does the pretrained network help on all of them?

5 Exercise 2: ECG heartbeats (1D)

Data: the MIT-BIH Arrhythmia Database: 44 half-hour ECG recordings, each from a different patient (a *record*), cut into heartbeats of 187 samples at 125 Hz, each scaled to the range 0–1. Every beat has one of four classes: N (normal), S (supraventricular), V (ventricular) or F (fusion). Every beat also remembers its record number. About 90% of the beats are class N.

Folder: `exercise2_ecg_1d/`

Step 1: split per patient (`01_prepare_data.py`)

Checks the environment, shows the shapes and the class balance, and plots example beats of each class.

Beats of the same patient look alike (same heart, same electrodes, same noise). If you split the beats at random, beats of the same patient end up in both the training and the validation set, and a model can score well just by recognising the patient. A real model is used on *new* patients, so whole records must go to only one of train, validation and test.

Your task

Decide how many of the 44 records go to train and to validation (about 70% / 15% / 15%; the rest is the test set) by filling in `N_TRAIN` and `N_VAL`. The script checks that no record is in two sets and prints the number of beats per class in each set.

Question

Do all three sets have beats of every class? Class F comes almost entirely from two patients: how many F beats does the test set have, and what does that mean for the reliability of its F1 score? Why is it harder to balance a split per patient than a split per beat?

Step 2: the 1D CNN (`ecg_cnn.py`, `02_train_cnn.py`)

The model follows Kachuee et al. (2018): a Conv1d, then five times (residual block, then `MaxPool1d`), then Flatten and two Linear layers. A beat has shape $(1, 187)$: one channel and 187 time samples. The residual block is given; you fill in the numbers of the layers around it.

Your task

In `ecg_cnn.py`: (1) fill in the five numbers used in all blocks: the number of channels (32 in the paper), the kernel size of the convolutions (5 in the paper), the padding that keeps the length unchanged, and the kernel size and stride of the pooling (stride 2 in the paper); (2) fill in the input sizes of the two Linear layers. The shape after each layer is written next to it as channels@length. Check your numbers with `python ecg_cnn.py`, which prints the shape after every layer.

Question

Which padding keeps the length of a convolution with kernel size k unchanged? How many features does Flatten output, and why? How many parameters has one residual block?

Then run `02_train_cnn.py` (CPU, then GPU). The network has 53,924 trainable parameters. After training, the script prints for the *validation patients* the precision, recall and F1 of every class. Look at the rare classes S and F, not only at the accuracy.

Step 3: the wrong split (`03_train_random_split.py`)

Trains the same model, but splits the beats of the training records at random into train and validation. Beats of the same patient are now in both. Nothing to fill in. The point is not to find the better model: it is to see that this validation set gives a too optimistic estimate, because the model is validated on patients it has already seen.

Step 4: evaluation (`04_evaluate.py`)

Evaluates both models once on the test records: whole patients that neither model saw. It prints the validation and test accuracy of each model and their difference (the *gap*), the precision, recall and F1 per class, the macro F1, and a confusion matrix per model.

Question

Compare the gap between validation and test accuracy for the two splits. What does it tell you about the validation set of the random split? Which classes have the lowest recall and precision on the test patients, and why is the accuracy a poor summary here? What accuracy would a model get that always answers "N"?

6 Summary of what to run

```
module load PyTorch-bundle/2.9.1-foss-2025a-CUDA-12.8.0-whl

cd exercise1_image_2d
python 01_prepare_data.py
python 02_train_scratch_cnn.py      # fill in the TODOs; DEVICE = "
    cpu", then "cuda"
python inspecting_resnet18.py
python mini_resnet.py               # fill in the TODOs
python 03_train_pretrained.py       # fill in the TODO; DEVICE = "
    cpu", then "cuda"
python 04_compare_and_evaluate.py

cd ../exercise2_ecg_1d
python 01_prepare_data.py           # fill in N_TRAIN and N_VAL
python ecg_cnn.py                   # fill in the TODOs
python 02_train_cnn.py              # DEVICE = "cpu", then "cuda"
python 03_train_random_split.py     # DEVICE = "cpu", then "cuda"
python 04_evaluate.py
```

References

- Kachuee, Fazeli, Sarrafzadeh. *ECG Heartbeat Classification: A Deep Transferable Representation.* 2018. https://arxiv.org/abs/1805.00794
- Moody, Mark. *The MIT-BIH Arrhythmia Database.* PhysioNet. https://physionet.org/content/mitdb/1.0.0/
- Krizhevsky. *Learning Multiple Layers of Features from Tiny Images* (CIFAR-10). https://www.cs.toronto.edu/~kriz/cifar.html